



5. Estruturas condicionais e de repetição

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Links e mídias	Pasta "Aula 5"
 Última atualização	@17 de novembro de 2025 03:59
 Status	Done
 Data início	@07/11/2025
 Data conclusão	@08/11/2025 19:58
 Esforço	Fácil

Anotações

Vídeo 1 - Indentação e blocos

Etapa 1: Indentação e os blocos de comandos

Vídeo 2 - Estruturas condicionais

Etapa 1: `if` / `if-else` / `elif`

Etapa 2: `if` aninhado

Etapa 3: `if` ternário

Vídeo 3 - Estruturas de repetição

Etapa 1: O que são estruturas de repetição?

Etapa 2: Comando `for` e a função incorporada `range`

Etapa 3: Comando `while`

Exercícios

Arquivos

Anotações

Vídeo 1 - Indentação e blocos

Etapa 1: Indentação e os blocos de comandos

Indentar o código é uma forma de manter o código legível e de fácil manutenção. Em Python, porém, ela exerce também outro papel: com ela, o interpretador consegue identificar onde um bloco de comando inicia e termina.

Lembre-se que, sem a indentação, o código não funciona!

Linguagens de programação costumam usar caracteres ou palavras reservadas para determinar o início e o fim de um bloco de código, em Java e C, por exemplos usam-se chaves.

A **prática recomendada** é utilizar **quatro espaços** para indentação.

Vídeo 2 - Estruturas condicionais

Estruturas condicionais possibilitam controlar o fluxo do código ao atender determinadas expressões lógicas.

Etapa 1: `if` / `if-else` / `elif`

Para criar uma estrutura condicional simples, com um único desvio de fluxo, podemos usar a palavra reservada `if`. O comando, então, testa a expressão

lógica e, em caso de retorno `True`, as ações presentes no bloco de código `if` são executadas.

```
saldo = 2000.0
saque = float(input('Informe o valor do saque: '))

if saldo >= saque:
    print('Realizando saque.')

if saldo < saque:
    print('Saldo insuficiente.')
```

Para criar uma estrutura condicional com dois desvios de fluxo, podemos usar as palavras reservadas `if` e `else`. O comando, então, testa a expressão lógica e, em caso de retorno `True`, as ações presentes no bloco de código `if` são executadas, caso contrário, são executadas as ações presentes no bloco de código `else`.

```
saldo = 2000.0
saque = float(input('Informe o valor do saque: '))

if saldo >= saque:
    print('Realizando saque.')
else:
    print('Saldo insuficiente.')
```

Em alguns cenários, precisamos de mais de dois desvios; para isso usamos a palavra reservada `elif`. Ele é composto de uma nova expressão lógica que será testada e, caso retorne `True`, o bloco de código correspondente será executado. Pode-se usar quantos `elif` forem necessários, porém deve-se **evitar criar uma estrutura condicional muito grande**, pois isso aumenta a complexidade do código.

```
opcao = int(input('Informe uma opção: [1] Sacar \n[2] Extrato: '))

if opcao == 1:
    valor = float(input('Informe a quantia para o saque: '))
elif opcao == 2:
```

```
print('Exibindo o extrato...')
else:
    sys.exit('Opção inválida!')
```

Etapa 2: **if** aninhado

Podemos criar estruturas condicionais aninhadas, para isso basta adicionar estruturas `if / elif / else` dentro do bloco de código de `if / elif / else`.

```
if conta_normal:
    if saldo >= saque:
        print('Saque realizado com sucesso!')
    elif saldo <= (saldo = cheque_especial):
        print('Saque realizado com uso do cheque especial!')
    else:
        print('Não foi possível realizar o saque: saldo insuficiente.')
elif conta_universitaria:
    if saldo >= saque:
        print('Saque realizado com sucesso!')
    else:
        print('Não foi possível realizar o saque: saldo insuficiente.')
else:
    if conta_vip:
        if saldo >= saque:
            print('Saque realizado com sucesso!')
        else:
            print('Não foi possível realizar o saque: saldo insuficiente. \nFale co
m seu gerente a qualquer hora: (11) 99999-8888.')
    elif conta_black:
        if saldo >= saque:
            print('Saque realizado com sucesso!')
        else:
            print('Saque realizado com sucesso! \nPorém, houve uso do cheque
especial, fale com seu gerente a qualquer hora: (11) 99999-8888.')
```

Etapa 3: **if** ternário

Este tipo de `if` possibilita escrever uma condição em uma única linha. Ele é composto por três partes: o retorno caso a expressão retorne `True`, a expressão lógica e o retorno caso a expressão retorne `False`.

```
status = "Sucesso" if saldo >= saque else "Falha"

print(f'{status} ao realizar o saque.')
```

Vídeo 3 - Estruturas de repetição

Etapa 1: O que são estruturas de repetição?

Usamos estruturas de repetição para repetir um trecho de código uma determinada quantidade de vezes. Podemos determinar essa quantidade de vezes previamente ou por meio de uma expressão lógica.

Etapa 2: Comando `for` e a função incorporada `range`

Usamos o comando `for` para percorrer um objeto iterável. Faz sentido usar `for` quando sabemos o número exato de vezes que um bloco de código deve ser executado, ou quando queremos percorrer um objeto iterável.

Por exemplo, a string `"Python"` itera em cada uma de suas letras: `P` - `y` - `t` - `h` - `o` - `n`.

```
texto = input('Informe um texto: ')
VOGAIS = 'AEIOU'

for letra in texto:
    if letra.upper() in VOGAIS:
        print(letra, end="")

print() # adiciona uma quebra de linha
```

Também é possível usar `for / else`:

```
texto = input('Informe um texto: ')
VOGAIS = 'AEIOU'
```

```
for letra in texto:
    if letra.upper() in VOGAIS:
        print(letra, end="")
    else:
        print() # adiciona uma quebra de linha
```

Usamos a função `range`, que acompanha o Python na instalação, para gerar uma sequência de números inteiros a partir de um valor inicial (inclusive) até um valor final (exclusive). Ao usar a expressão `range(i, j)`, Python exibe `i, i+1, i+2, i+3, ..., j-1`. Esta função recebe três parâmetros: `stop` (obrigatório, onde parar), `start` (opcional, onde começar) e `step` (opcional, de quantos em quantos contar).

```
range(4) # exibe range(0, 4), indicando que é um objeto do tipo "range"
list(range(4)) # exibe a lista [0, 1, 2, 3], lembrando que o índice em Python i
               # nicia do 0
```

Como utilizar o `range` com o `for`:

```
for numero in range(0, 11):
    print(numero, end=" ")

# Exibir a tabuada do 8
for numero in range(0, 81, 8):
    print(numero, end=" ")
```

Etapas 3: Comando `while`

Usamos o comando `while` para repetir um bloco de código várias vezes. Faz sentido usá-lo quando não se sabe a quantidade exata de vezes que um bloco de código deve ser executado.

```
opcao = -1

while opcao != 0:
    opcao = int(input("[1] Sacar\n[2] Extrato\n[0] Sair: "))

    if opcao == 1:
        print("Sacando...")
```

```
elif opcao == 2:  
    print("Exibindo o extrato")
```

break : interrompe a execução do bloco de código em **while** ou **for** caso determinada condição seja atendida.

Lembrando que é possível criar um loop infinito usando **while True** , que só interrompe caso a expressão que gere o **break** seja satisfeita. Então, qualquer expressão usada com **while** que seja **True** gera um loop infinito, por exemplo **1 == 1** .

continue : interrompe a execução do bloco de código em **while** ou **for** caso determinada condição seja atendida, mas continua executando após a condição.

Exercícios

100% ⇒ 7/7 corretas

Arquivos

Pasta "Aula 5"